
📄 Payroll Management System – Architecture & PoC Documentation

---

1. 📌 Project Overview

Project Name: Payroll Management System
Type: REST API + Angular Frontend

Tech Stack:

- Backend: .NET 8/9 (ASP.NET Core Web API)
- ORM: Entity Framework Core (+ optional Dapper)
- Frontend: Angular
- Database: SQL Server

Objective:
To design and implement a secure, scalable payroll system that manages employees, salaries, and leave workflows with role-based and hierarchical access control.

---

2. 👥 User Roles & Hierarchy

🔐 Admin

- Full system access
- Manage employees and salary records
- Assign managers to employees
- Approve/reject leave requests for managers
- View all system data

👩‍💼 Manager

- View own profile
- View salary details
- View employees under supervision
- Approve/reject leave requests for employees
- View team leave records
- Apply for leave and view leave status

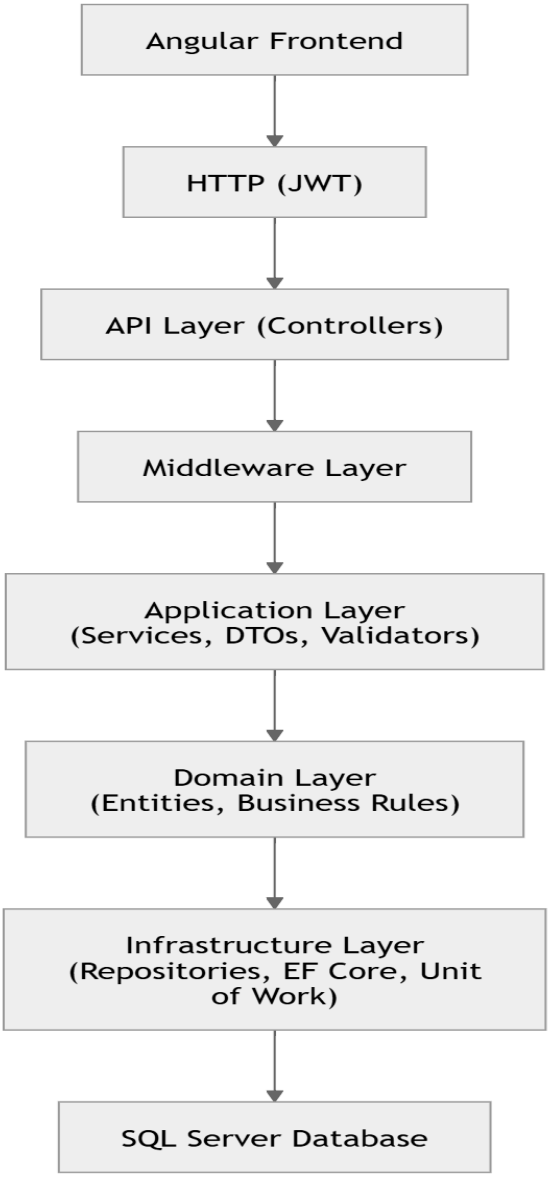
Employee

- View own profile
- View salary details
- Apply for leave
- View leave status

---

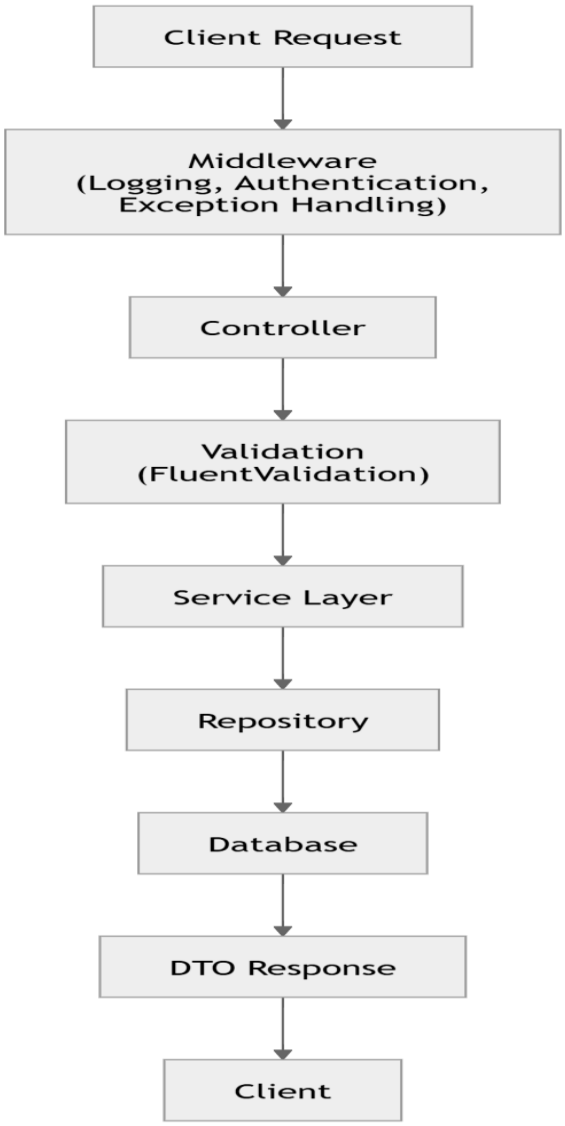
3. System Architecture

High-Level Architecture



---

4. 🔄 Request Lifecycle



---

5. 🔐 Authentication & Authorization

- JWT-based authentication
- Access token with 20-minute expiry
- Refresh token stored in database
- Role-based authorization (Admin, Manager, Employee)

---

6. 🧩 Functional Requirements

👩‍💼 Employee Management

Admin: Create, update, delete employees, assign managers, view all employees
Manager: View own profile and team members
Employee: View own profile

💰 Payroll Management

Admin: Define salary structure (Basic, HRA, Allowances, Deductions), generate payslips
Manager / Employee: View salary details, download payslip

📝 Leave Management

Approval Rules

Applicant	Approved By
Employee	Manager
Manager	Admin

Access Rules

- Managers handle leave requests of their team members
- Admin handles leave requests submitted by managers
- Self-approval is prohibited

📊 Common Features

- Pagination
- Filtering
- Sorting

---

7. 🗃️ Database Design

Users

- Id, Username, Email, PasswordHash, RoleId

Roles

- Id, Name (Admin / Manager / Employee)

Employees

- Id, UserId, FullName, Department, Designation, DateOfJoining, ManagerId

Salaries

- Id, EmployeeId, Basic, HRA, Allowances, Deductions, NetSalary

Leaves

- Id, EmployeeId, FromDate, ToDate, Reason, Status, ApprovedBy, ApproverRole

RefreshTokens

- Id, UserId, Token, ExpiryDate

---

8. 🔧 Technical Architecture

Design Patterns: Repository, Unit of Work, Dependency Injection
Layered Structure: Controller → Service → Repository → Database
DTO Strategy: Separate DTOs, no direct entity exposure, role-based shaping
Validation: FluentValidation
Exception Handling: Global middleware
Logging: Request, response, error, and activity tracking
Security: CORS, input validation, secure password hashing

---

9. 🔐 Authorization Strategy

Role	Scope
Admin	Full system
Manager	Team data
Employee	Own data

Data Access Rule: Managers can access only employees where Employee.ManagerId = LoggedInManagerId

---

10. ✒️ API Design

Authentication: POST /api/auth/login, POST /api/auth/refresh
Employees:

- Admin: GET /api/employees
- Manager: GET /api/employees/team
- Employee: GET /api/employees/me

Salaries:

- Admin: POST /api/salaries
- Employee/Manager: GET /api/salaries/my

Leaves:

- Employee: POST /api/leaves, GET /api/leaves/my
- Manager: GET /api/leaves/team, PUT /api/leaves/{id}/approve, PUT /api/leaves/{id}/reject
- Admin: GET /api/leaves, PUT /api/leaves/{id}/approve, PUT /api/leaves/{id}/reject

---

11. 🔦 Testing

- Unit testing for service layer
- Controller testing

---

12. 📝 Documentation

- Swagger / OpenAPI
- API Versioning

---

13. ⚡ Performance Considerations

- Async/await for all operations
- Pagination for large datasets
- Optimized queries
- Optional caching

---

14. 🅰 Frontend (Angular)

Pages: Login, Dashboard, Employee List, Leave Management, Salary View
Features: JWT token handling, HTTP interceptor, role-based UI, form validation

---

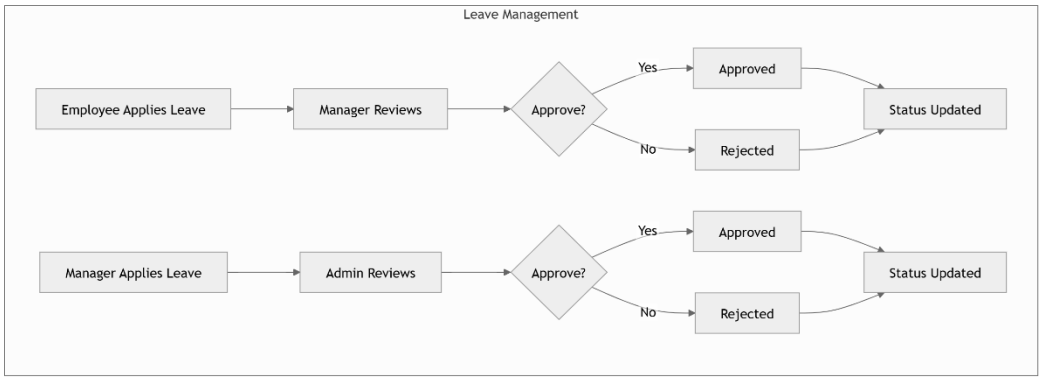
15. ⏱ Timeline

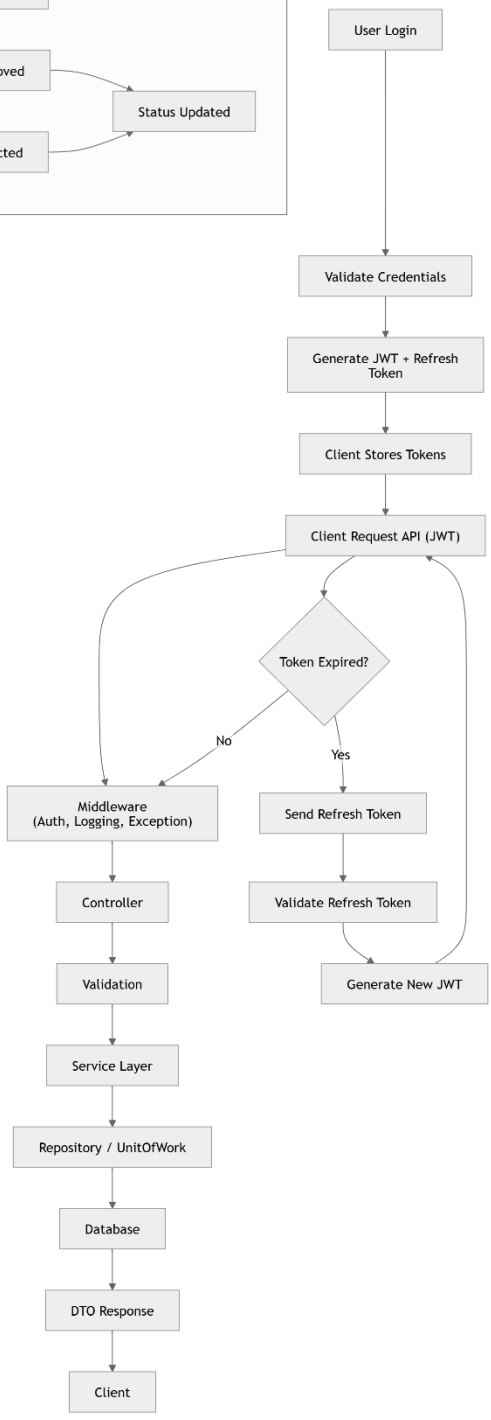
Week 1: Setup, database, CRUD APIs, Authentication, roles
Week 2: Leave workflow, middleware, security, Angular UI, testing

---

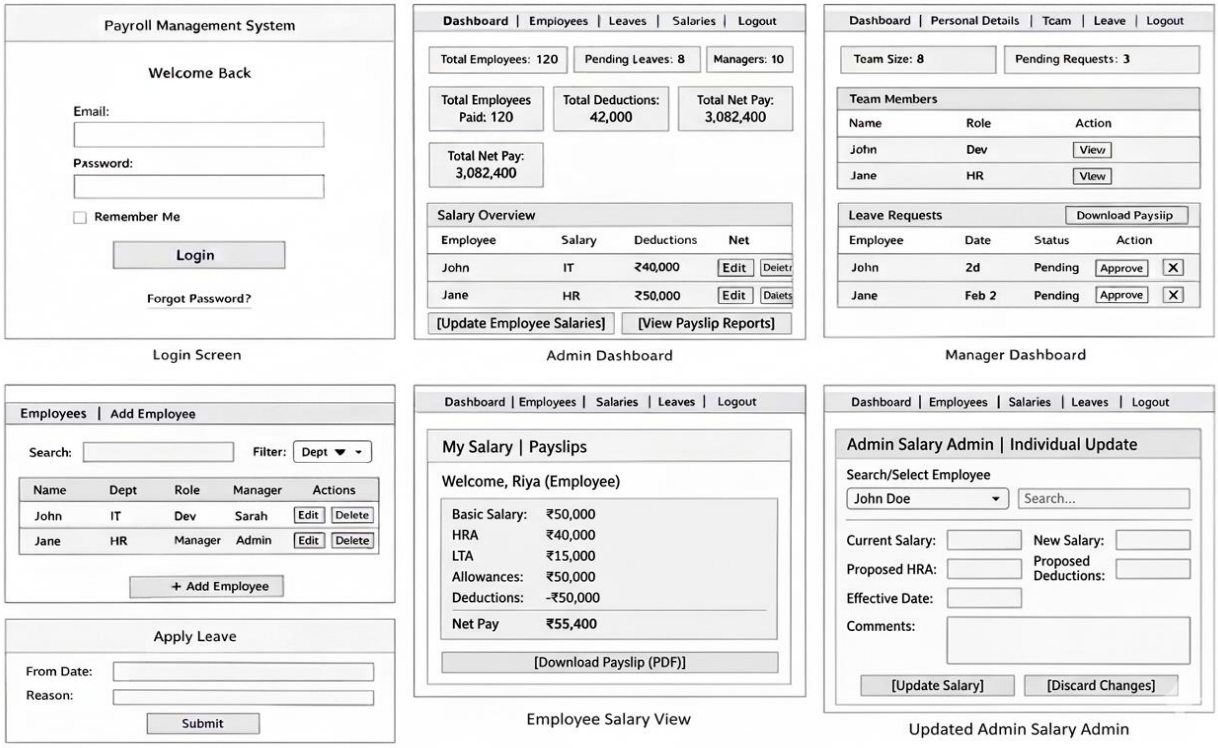
16. Visual Diagrams

16.1 Workflow Diagram

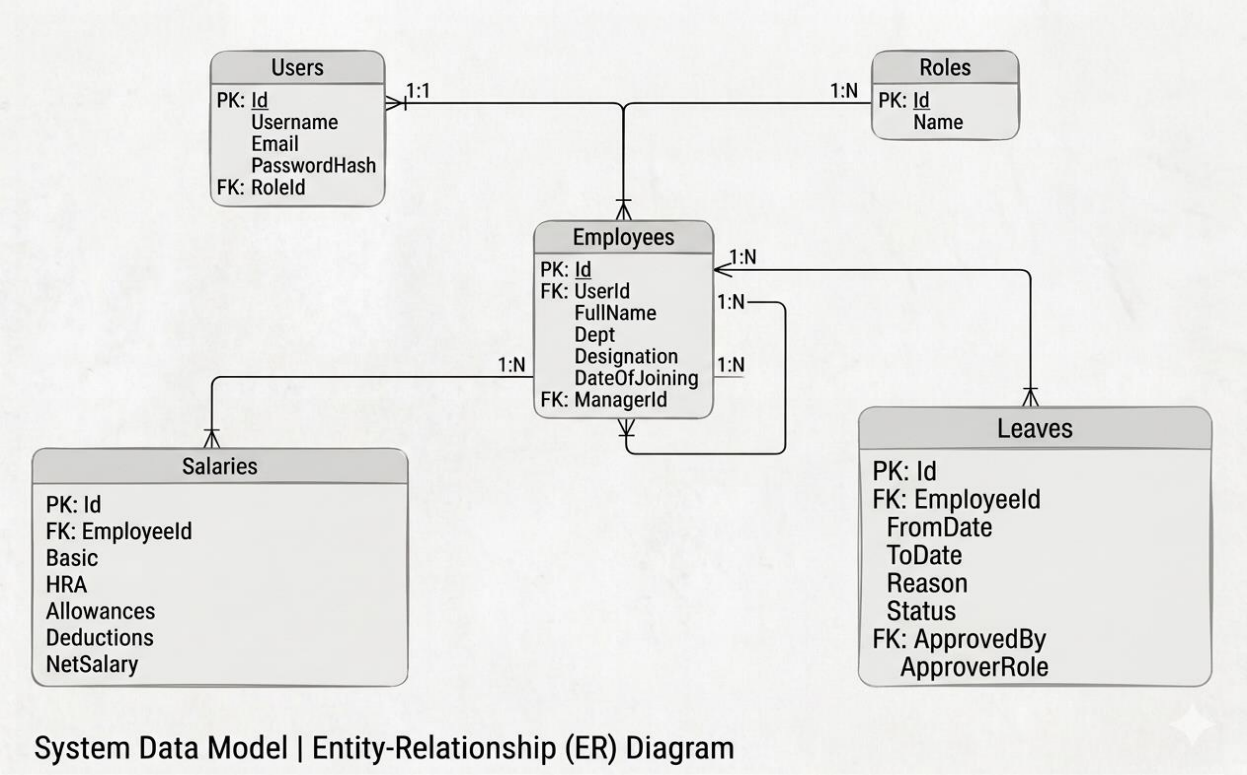




16.2 Dashboard Diagram



16.3 ER Diagram



16.3 Leave Approval Workflow (Sequence Diagram)

